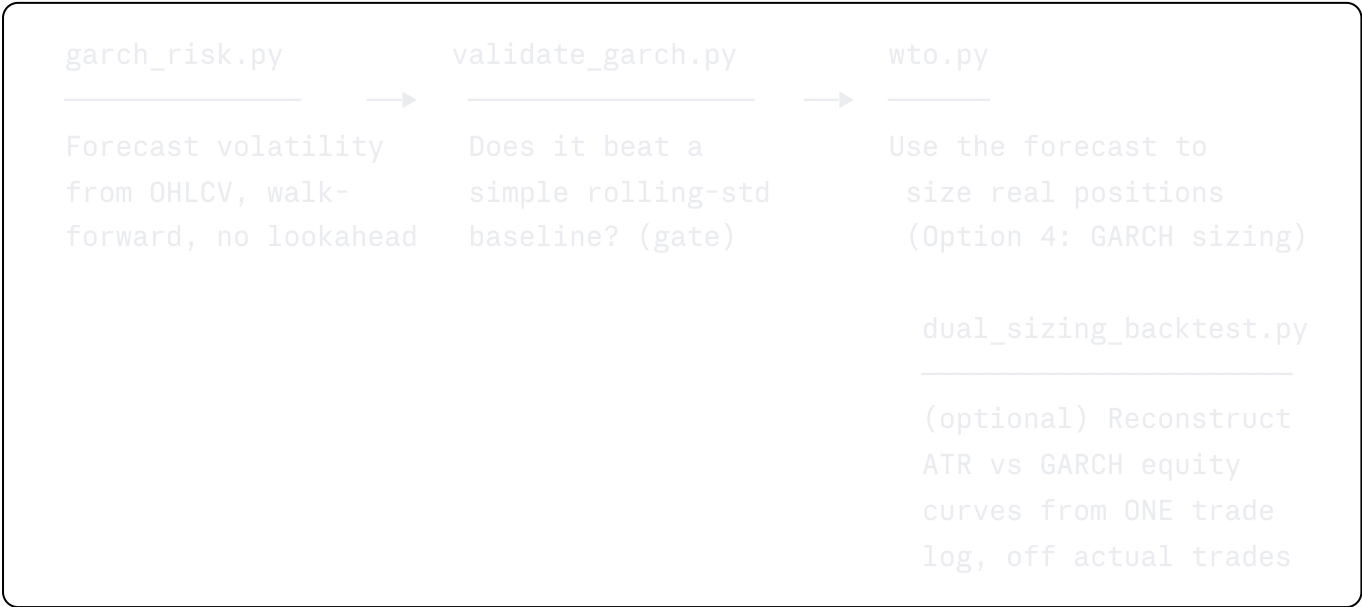


GARCH Volatility Pipeline — Documentation

A standalone volatility-forecasting pipeline (GARCH / GJR-GARCH / EGARCH, walk-forward, no-lookahead) with its own validation tooling, designed to plug into `wto.py`'s position sizing as an alternative to static ATR.

Pipeline overview



Four scripts, four jobs:

Script	Job	Required before running
<code>garch_risk.py</code>	Produce a <code>GarchVol</code> forecast CSV from an OHLCV file	Nothing — first step
<code>validate_garch.py</code>	Score that forecast against a rolling-std baseline	A <code>garch_risk.py</code> output
<code>wto.py</code>	Use <code>GarchVol</code> for real position sizing, run the full 4-stage pipeline	A validated <code>garch_risk.py</code> output (recommended)
<code>dual_sizing_backtest.py</code>	Compare ATR vs GARCH sizing off one trade log, no re-run needed	A <code>wto.py</code> trade log export + <code>garch_risk.py</code> output

Step 1 — Forecast: `garch_risk.py`

Produces `<file>_<model>.csv` with a `Date, GarchVol` column — `GarchVol` is a **fractional daily-return volatility forecast** (e.g. `0.018` = 1.8%), not a price distance.

Full CLI reference

Flag	Default	Meaning
<code>--file</code>	<i>required</i>	OHLCV CSV to forecast on
<code>--output</code>	<code><file>_<model>.csv</code>	Output path
<code>--model</code>	<code>egarch</code>	<code>garch</code> <code>gjr</code> <code>egarch</code>
<code>--refit-every</code>	<code>20</code>	Bars between parameter refits
<code>--min-train</code>	<code>250</code>	Minimum bars before first fit
<code>--window-days</code>	<code>550</code>	Rolling training window (bars). <code>0</code> = unbounded/expanding — avoid this; a single shock inside an expanding window permanently corrupts every later refit (see Gotchas)
<code>--dist</code>	<code>normal</code>	<code>normal</code> <code>t</code> <code>skewt</code>
<code>--scale</code>	<code>100.0</code>	Internal numerical-stability scaling — leave as default
<code>--clip</code>	<code>7.0</code>	Clip daily returns to $\pm$ this percent before fitting/forecasting. <code>0</code> disables
<code>--quiet</code>	off	Suppress per-refit progress printing
<code>--start-date</code> / <code>--end-date</code>	none	Restrict input date range

Timeframe scaling — the most common mistake

Every default above is tuned for **daily bars**. On any other bar size, scale by bars-per-trading-day:

Timeframe	Bars/day (approx)	<code>--min-train</code>	<code>--refit- every</code>	<code>--window- days</code>
Daily	1	250	20	550
4h (24h asset, e.g. XAUUSD)	~6	1500	120	3300
4h (equity, ~5.5 trading days/wk)	~4-5	adjust proportionally	adjust proportionally	adjust proportionally

Rule of thumb: multiply every one of these three flags by the same bars-per-day factor. Using daily defaults on 4h data silently trains on ~6x less real history and refits ~6x too often.

Example commands

```
bash

# Daily equity, GJR model (recommended default for equities)
python garch_risk.py --file DLF-EQ.csv --model gjr

# XAUUSD 4h, properly scaled
python garch_risk.py --file xauusd_4h.csv --model gjr \
    --min-train 1500 --refit-every 120 --window-days 3300

# XAUUSD 1d
python garch_risk.py --file xauusd_1d.csv --model gjr \
    --min-train 250 --refit-every 20 --window-days 550

# Compare all three models on the same file (run 3x, different --model)
python garch_risk.py --file TCS-EQ.csv --model garch
python garch_risk.py --file TCS-EQ.csv --model gjr
python garch_risk.py --file TCS-EQ.csv --model egarch
```

Step 2 — Validate: `validate_garch.py`

Scores `GarchVol` against a simple rolling-std baseline on two metrics — correlation with next-day realized `[return]`, and mean absolute error. **Only trust a result that wins both metrics**, not just one.

Full CLI reference

Flag	Default	Meaning
<code>--ohlcv</code>	<i>required</i>	Original OHLCV CSV
<code>--garch</code>	<i>required</i>	<code>garch_risk.py</code> output CSV
<code>--atr-window</code>	<code>14</code>	Baseline rolling-std lookback — must match what <code>wto.py</code> actually uses for ATR (or its bar-scaled equivalent), or the comparison isn't fair
<code>--start-date</code> / <code>--end-date</code>	<i>none</i>	Restrict scored range — use for split-sample checks

Split-sample checking (do this before trusting any single pass/fail)

```
bash

# Full range
python validate_garch.py --ohlcv DLF-EQ.csv --garch DLF-EQ_gjr.csv --atr-window

# First half
python validate_garch.py --ohlcv DLF-EQ.csv --garch DLF-EQ_gjr.csv --atr-window

# Second half
python validate_garch.py --ohlcv DLF-EQ.csv --garch DLF-EQ_gjr.csv --atr-window
```

To find a fair split point (accounting for `--min-train` burn-in, so you don't split into an empty first half):

```
bash

python -c "
import pandas as pd
df = pd.read_csv('DLF-EQ_gjr.csv', index_col=0, parse_dates=True)
valid = df['GarchVol'].dropna()
start, end = valid.index[0], valid.index[-1]
mid = start + (end - start) / 2
print('Valid range:', start.date(), '->', end.date())
print('Midpoint   :', mid.date())
"
```

Step 3 — Use it for real sizing: wto.py

The key discovery this integration is built around

wto.py's original next() called self.buy(sl=sl, tp=tp) with no size= argument — meaning every trade used backtesting.py's default (near-full-equity) sizing, regardless of risk_pct. The patched version adds a sizing_mode that actually implements risk-based sizing for the first time, for both ATR and GARCH:

sizing_mode	Stop distance from	Position size
legacy (default)	ATR	Full equity (original, unchanged behavior)
atr	ATR	risk_pct-based (NEW — real baseline)
garch	GarchVol × close_price	risk_pct-based (Option 4)

Full CLI reference (new flags only — see wto.py --help for the rest)

Flag	Default	Meaning
--sizing-mode	legacy	legacy atr garch
--garch-file	none	garch_risk.py output CSV — required when --sizing-mode garch
--garch-sl-mult	1.5	Stop-loss multiple applied to GarchVol (mirrors --atr-sl-mult / atr_sl_mult)
--dual-compare	off	After a --sizing-mode garch --pipeline run, also re-run the exact same combo/params under sizing_mode='atr' and print a side-by-side comparison table

Example commands

```
bash
```

```
# Full 4-stage pipeline with GARCH sizing + ATR comparison table at the end
python wto.py --file DLF-EQ.csv --pipeline \
    --sizing-mode garch --garch-file DLF-EQ_gjr.csv --dual-compare

# Same, but with the new ATR-risk baseline (no GARCH) – useful to confirm
# the "atr" mode alone (real risk-based sizing) behaves sensibly first
python wto.py --file DLF-EQ.csv --pipeline --sizing-mode atr

# Original behavior, completely unchanged (no new flags passed)
python wto.py --file DLF-EQ.csv --pipeline

# Single combo, single run, GARCH sizing – for quick checks without the
# full Stage 1-3 search
python wto.py --file DLF-EQ.csv --single-run --combo use_rsi,use_macd \
    --sizing-mode garch --garch-file DLF-EQ_gjr.csv
```

When the data-range prompt appears (Choose (1, 2 or 3)), pick **3 (6+3 walk-forward)** for anything you intend to trust — it optimizes on 6 years and tests once, untouched, on the remaining 3. Options 1 and 2 give Stages 1-3 the whole dataset to search over, with no genuine holdout.

Step 4 (optional) — Fast ATR-vs-GARCH comparison without a re-run:

dual_sizing_backtest.py

Since entries/exits are identical regardless of sizing method, this reconstructs **two equity curves from one existing trade log** — faster than running wto.py twice, and guarantees no parameter drift between the two comparisons.

Full CLI reference

Flag	Default	Meaning
<code>--trades</code>	<i>required</i>	Trade log CSV, e.g. <code>stats._trades.to_csv('trades.csv')</code> from a <code>wto.py</code> / <code>backtesting.py</code> run
<code>--ohlcv</code>	<i>required</i>	OHLCV CSV with an <code>ATR</code> column
<code>--garch</code>	<i>required</i>	<code>garch_risk.py</code> output CSV
<code>--equity</code>	<code>100000</code>	Starting equity (both methods)
<code>--risk-pct</code>	<code>0.01</code>	Risk per trade as fraction of equity
<code>--atr-sl-mult</code>	<code>2.0</code>	Stop-loss multiple applied to ATR/GarchVol
<code>--periods-per-year</code>	<code>252</code>	For annualizing Sharpe/Sortino — <code>252</code> for daily bars, <code>~1565</code> for 4h bars
<code>--contract-size</code>	<code>100</code>	Units per 1.0 lot. Use <code>1</code> for equities (whole shares), <code>100</code> for XAUUSD (oz/lot, standard spec)
<code>--lot-step</code>	<code>0.01</code>	Minimum lot increment. Use <code>1</code> for equities
<code>--min-lot</code>	<code>0.01</code>	Minimum tradeable size. Use <code>1</code> for equities

Example commands

bash

```
# Equity (DLF) — whole-share sizing, no fractional/lot system needed
python dual_sizing_backtest.py --trades dlf_trades.csv --ohlcv DLF-EQ.csv \
    --garch DLF-EQ_gjr.csv --equity 100000 --risk-pct 0.01 --atr-sl-mult 2.0 \
    --periods-per-year 252 --contract-size 1 --lot-step 1 --min-lot 1

# XAUUSD — real lot-based sizing (standard MT5-style contract spec)
python dual_sizing_backtest.py --trades xau_trades.csv --ohlcv xauusd_4h.csv \
    --garch xauusd_4h_gjr.csv --equity 100000 --risk-pct 0.01 --atr-sl-mult 2.0 \
    --periods-per-year 1565 --contract-size 100 --lot-step 0.01 --min-lot 0.01
```

Recommended end-to-end workflow

bash

```
# 1. Forecast
python garch_risk.py --file DLF-EQ.csv --model gjr

# 2. Validate - full range, then split-sample
python validate_garch.py --ohlc DLF-EQ.csv --garch DLF-EQ_gjr.csv --atr-window
python validate_garch.py --ohlc DLF-EQ.csv --garch DLF-EQ_gjr.csv --atr-window
python validate_garch.py --ohlc DLF-EQ.csv --garch DLF-EQ_gjr.csv --atr-window

# 3. Only if BOTH halves pass on BOTH metrics - run the real pipeline
python wto.py --file DLF-EQ.csv --pipeline \
    --sizing-mode garch --garch-file DLF-EQ_gjr.csv --dual-compare
# choose option 3 (6+3 walk-forward) at the prompt

# 4. (optional) Fast re-check on the resulting trade log
python dual_sizing_backtest.py --trades trades.csv --ohlc DLF-EQ.csv \
    --garch DLF-EQ_gjr.csv --contract-size 1 --lot-step 1 --min-lot 1
```

Gotchas (found the hard way — don't repeat these)

- **Expanding window** (`--window-days 0`): a single extreme shock (e.g. an election-result crash) never ages out and permanently corrupts every later refit. Always use a bounded rolling window in production.
- **EGARCH forecasting**: requires `method='simulation'`, not the default analytic method — analytic forecasting is not valid for EGARCH's log-variance recursion and silently produces garbage (already handled inside `garch_risk.py`) — just don't hand-roll your own EGARCH forecast call elsewhere without the same fix).
- **Baseline window mismatch**: `--atr-window` in `validate_garch.py` must match `wto.py`'s real ATR period (bar-scaled if not daily) — using a wider baseline window than production makes GARCH look artificially better; a narrower one makes it look artificially worse.
- **Warm-up bias in split-sample tests**: the training window doesn't reach full size until `(min_train + window_days)` bars in — scoring a split that includes this ramp-up period unfairly penalizes the "warming up" half. Exclude it, or split only the post-warm-up range.
- **Unit mismatch**: `GarchVol` is a fractional return (`0.018`), ATR is a price distance. Always multiply `(GarchVol × close_price)` before comparing/combining it with ATR — this is handled inside `wto.py`'s patched `next()`, but matters if you write any other integration.
- **Small samples produce false verdicts in both directions**: a "clean pass" or "clean fail" on a few hundred bars can flip on a larger sample. Don't trust a single validation run without checking it holds on a longer history and a split-sample test.